Promo Especial
Precio especial por tiempo limitado
$287.929 Ahorras $71.971
Antes ~~$359.900~~ | Paga en 4 cuotas
Quiero este precio
Solo quedan: 1d 10h 57m 59s

Blog

ECMAScript 6: nueva sintaxis y características para JavaScript

10 Actualizado el 24 de abril de 2015 | 8 min de lectura

ESCRITO POR
sergiodxa 21,771 Pts

Hasta ahora en JavaScript si uno quería programar orientado a objetos lo que debía hacer era aprovecharse de la herencia de prototipos y así poder crear las “clases”. Para lograrlo usábamos una sintaxis similar a esta:

```
function Persona(nombre, edad) {
  this.nombre = nombre;
  this.edad   = edad;
}
Persona.prototype.presentarse = function() {
  return 'Hola me llamo ' + this.nombre + ' y tengo ' + this.edad + ' años';
}

var Sergio = new Persona('Sergio', 22);
Sergio.presentarse(); // 'Hola me llamo Sergio y tengo 22 años'
```

Una pequeña variación podía ser **definir la función presentarse dentro de Persona usando**:

```
function Persona(nombre, edad) {
  this.nombre = nombre;
  this.edad   = edad;

  this.presentarse = function () {
    return 'Hola me llamo ' + this.nombre + ' y tengo ' + this.edad + ' años';
  }
}
```

> Aprende más sobre este tema en la nueva Escuela de JavaScript de Platzi 🚀

Esta fue siempre la forma de hacer POO en JavaScript, y aunque los prototipos son muy poderosos para los programadores que vienen de lenguajes orientados a objetos, como Java o PHP, siempre les ha resultado extraño, ya que no son propiamente objetos, o al menos no como el resto de lenguajes considera los objetos.

Nota: **el soporte para clases de ECMAScript 6 en los navegadores actualmente es nulo**, todavía está en desarrollo y no está listo para ser usado, hay sin embargo formas de escribir código usando ES6 y luego convertirlo a ES5 con resultados bastante buenos,* *pero no es un soporte nativo, sigue siendo ES5**.

Nueva sintaxis

Para hacer más fácil esta tarea en ECMAScript 6 se va a agregar una forma de crear clases, como en los demás lenguajes, aunque al final esta nueva sintaxis para hacer POO es simplemente una capa sobre la sintaxis actual, mediante prototipos, pero más fácil de entender.

En la nueva sintaxis, si quisiéramos hacer el mismo ejemplo de antes escribiríamos:

```
class Persona {
  // constructor donde definir las variables que se reciben y guardarlas en el
objeto usando this
  constructor(nombre, edad) {
    this.nombre = nombre;
    this.edad   = edad;
  }

  // método para presentarse
  presentarse() {
    return 'Hola me llamo ' + this.nombre + ' y tengo ' + this.edad + ' años';
  }
}
```

```
var Sergio = new Persona('Sergio', 22);
Sergio.presentarse();
```

Como podemos ver, la sintaxis es mucho más parecida a la de otros lenguajes (teniendo un constructor y métodos). Sin embargo, como dije antes, esta es una capa por encima del método actual, por esa razón al momento de instancia **clase** el resto de la sintaxis es igual.

Extender clases

Al igual que en otros lenguajes de programación, una clase puede extender otra clase heredando métodos o propiedades de la clase padre. Siguiendo el ejemplo anterior, vamos a extender la clase Persona:

```
class Desarrollador extends Persona {
  constructor(nombre, edad, cargo) {
    super(nombre, edad);
    this.cargo = cargo;
  }

  presentarse() {
    return super.presentarse() + ' y soy desarrollador ' + this.cargo;
  }
}

var Sergio = new Desarrollador('Sergio', 22, 'Frontend');
Sergio.presentarse(); // 'Hola me llamo Sergio y tengo 22 años y soy desarrollador Frontend'
```

Como se puede ver la sintaxis es: indicar el nombre de la nueva clase y que extiende a otra clase. Luego, si se desea extender algún método ya existente, como el constructor, se reciben los mismo parámetros que en la clase padre más los nuevos y luego se usa la función **super()**.

La función super() ejecuta el método con el mismo nombre desde el que se está llamando a super(), de esta forma al definir el nuevo constructor llamamos a super() y le pasamos los mismos parámetros que recibe el constructor de Persona, entonces se ejecuta ese constructor y luego código del nuevo.

En el caso del método presentarse usé **super.presentarse()**, esto lo que hace es ejecutar el método presentarse de la clase padre, en este caso sería lo mismo que hacer simplemente super(), pero si hubiese otro método con otro nombre que queremos ejecutar super.metodo(), nos permite llamar a un método de la clase padre desde cualquier otro método de la nueva clase.

Getters y Setters

En algunos lenguajes de programación (como Java) existen los getters y setters, que son lo que se llama **mutator method**, estos métodos que se usan para controlar variables internas de un objeto (propiedades). Para usarlos simplemente se agrega get o set delante del nombre del método de la siguiente forma:

```
class Persona {
  constructor(nombre, edad) {
    this.nombre = nombre;
    this.edad   = edad;
  }

  get verNombre() {
    return this.nombre;
  }
  set nuevoNombre(nuevo) {
    this.nombre = nuevo;
  }
}
var Sergio = new Persona('Sergio', 22);
Sergio.verNombre; // devuelve Sergio
Sergio.nuevoNombre('Daniel'); // cambia el valor de nombre a Daniel
Sergio.verNombre; // devuelve Daniel
```

Como se puede ver es bastante simple: definir un método get con el nombre que quieras (no puede ser el nombre de la propiedad) y este debería devolver el valor deseado (tecnicamente puede hacer cualquier cosa el método), o defines un método set con otro nombre (tampoco el mismo de la propiedad) y que recibe el nuevo valor y lo asigna a this (también puede hacer cualquier cosa en realidad, esto es útil para poder validar el nuevo valor).

Aunque esto hace bastante más legible y limpio el código, al tener métodos específicos para obtener o modificar propiedades del objeto, la verdad es que no son necesarios ya que simplemente usando la sintaxis de objetos de toda la vida puedes obtener el valor de una propiedad y modificarlo.

```
Sergio.nombre = 'Sergio'; // cambia el valor de nombre a 'Sergio'
Sergio.nombre; // Devuelve 'Sergio'
```

Métodos estáticos

Al igual que en otros lenguajes también va a ser posible crear métodos estáticos usando la palabra clave **static** antes del nombre del método.

```
class miClase {
  static miMetodo() {
    return 'hola mundo'
  }
}
```

Luego para poder usarlo simplemente llamas al método desde la clase sin instanciar:

```
let mensaje = miClase.miMetodo(); // 'hola mundo';
```

Características

Los nombres de las clases no pueden ser *eval* ó *arguments*; no están permitidos nombres de clase repetidos, el nombre *constructor* solo puede ser usado para métodos, no para getters, setter o un generador de métodos (otro características de ES6, quizá para otro tutorial).

Las clases no se pueden llamar antes de definirse.

```
new Persona(); // runtime error
class Persona {};
```

Aunque esto no debería ser un problema, todavía se puede instanciar la clase desde cualquier parte, solo es necesario esperar a que esté definida.

Si no hay método constructor, entonces el método por defecto va a ser:

```
constructor(arg1, arg2...) {
  super(arg1, arg2...);
}
```

Conclusión

Aunque la forma actual de hacer POO usando prototype es muy poderosa, esta nueva forma va a traer varios beneficios, ya sea que termine siendo más fácil para los principiantes hacer que crear una subclase o hacer código más portable entre frameworks facilitando la reutilización de código.

De todas formas, al final sigue siendo la clásica forma de crear objetos debajo de toda la nueva sintaxis.

Aprovecha el precio especial

Plan Expert

Un año de acceso con cursos ilimitados

Antes $359.900

$287.929/año

Ahorras $71.971

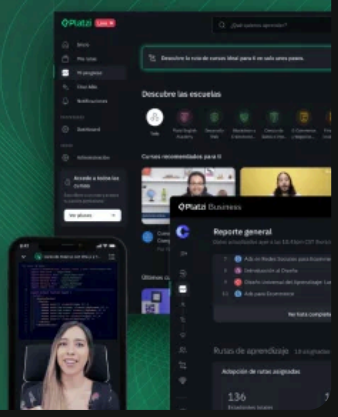
Suscríbete a Plan Expert

Paga a 4 cuotas sin intereses de $71.983

Educación online efectiva

Aprende con expertos de la industria

Crea tu cuenta sin costo



COMPARTE ESTE ARTÍCULO Y MUESTRA LO QUE APRENDISTE

0 Comentarios

Entra a Platzi para escribir tu comentario

Artículos relacionados



Qué roles hay en el UX design

El ámbito de la experiencia de usuario está compuesto por muchos roles profesionales diferentes y cada uno...



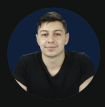
torresburriel

209



Platzi English Academy: Leave vs. live, pronunciation training

"I speak English, but I have difficulty understanding native speakers". Many of us have felt this way before, and this...



CesarCordero

180



¿Qué es una narrativa?

Desde el punto de estructura narrativ nuestra historia. N



paolamaz

179

Platzi

La escuela de tecnología de Latinoamérica

Aprende en nuestras redes

Certificadores oficiales en tecnologías

TOEFL Google aws

Reconocidos por

ASU+GSV SUMMIT